2.3.

```
void InsertSqList ( SqList &La, ElemType x) {
    if (La.length ==La.listsize)     Increment (La);
    int i=La.length-1;
    while (i>=0 && La.elem[i] > x) {
        La.elem[i+1] = La.elem[i];
        i--; }
    La.elem[i+1] = x;
    La.length ++;     }
```

2.5.

```
void ReverseSqList ( SqList &L ) {
    if (L.length <=1)   return;
    int mid = L.length /2;
    for (int i=0; i<mid ; i++) {
        ElemType temp = L.elem[i];
        L.elem[i] = L.elem[L.length-1-i];
        L.elem[L.length-1-i] = temp; }
```

2.6.

```
void ReverseLinkList_WithHeader ( LinkList &L) {
    if (L->next == NULL || L->next->next ==NULL)     return;
    LNode *p = L->next;
    LNode *q;
    L->next = NULL;
    while (p != NULL) {
        q = p->next;
        p->next = L->next;
        L->next = p;
        p= q; }     }
```

2.9.

```
void MergeLinkList ( LinkList &La, LinkList &Lb, LinkList &Lc) {
    LNode *pa = La->next;
    LNode *pb = Lb->next;
    LNode *q;
    Lc = La;
    Lc->next = NULL;
    while ( pa != NULL && pb != NULL) {
        if ( pa->data <= pb->data) {
            q= pa->next;
            pa->next = Lc->next;
            Lc->next = pa;
            pa = q;
        } else {
            q = pb->next;
            pb->next = Lc->next;
            Lc->next = pb;
            pb = q;
        } //end if-else     } //end while
    LNode *remaining = ( pa != NULL) ? pa : pb;
    while (remaining != NULL) {
        q= remaining->next;
        remaining->next = Lc->next;
        Lc->next = remaining;
        remaining = q; }     //end while
    delete Lb;
}
```

2.10.

```
void Delete_s_prior (LinkList &L) {
    LNode *p = s;
    LNode *q;
    while (p->next->next != s) p = p->next;
    q = p->next;
    p->next = s;          去除节点后前后接上
    delete q;
}
```

2.12.

```
void PartitionOddEven (LinkList &L) {
    if (L->next == NULL || L->next->next == NULL) return;
    LNode *p = L->next;
    LNode *q;
    LNode *head_even = new LNode;
    head_even->next = NULL;
    L->next = NULL;
    while (p) {
        q = p->next;
        if (p->data % 2 == 1) {
            p->next = L->next;
            L->next = p;
        } else {
            p->next = head_even->next;
            head_even->next = p; }
        p = q; }
    Lnode *tail_odd = L;
    while (tail_odd->next) tail_odd = tail_odd->next;
    tail_odd->next = head_even->next;
    delete head_even; }
```